

Subject 3: Token-Level Processing & Cost Optimization

1. The 80/20 Core (Hiring-Critical)

What Actually Matters (20% → 80% Impact)

Included (Production & Interview Critical):

1. **Token-Accurate Cost Tracking** (30% of value)
 - Per-request cost calculation (input + output tokens)
 - Real-time budget monitoring and alerts
 - Cost attribution (per user, feature, endpoint)
 - Model-specific tokenizer usage (tiktoken for OpenAI, etc.)
 - Why: Without tracking, LLM costs spiral out of control (\$100 → \$10,000/month)
2. **Context Window Management** (25% of value)
 - Truncation strategies (head, tail, middle, sliding window)
 - Priority-based content retention (system > examples > context)
 - Token budget allocation across prompt components
 - Why: Exceeding context limits = silent failures or crashes
3. **Prompt Compression Techniques** (20% of value)
 - Redundancy elimination without losing semantics
 - Example selection (dynamic few-shot)
 - Instruction condensation
 - Why: 50% token reduction = 50% cost savings at same quality
4. **Model Selection & Routing** (15% of value)
 - Cost/quality/latency tradeoff analysis
 - Task-based routing (GPT-4 for complex, GPT-3.5 for simple)
 - Fallback chains (try cheap first, escalate if needed)
 - Why: Right model selection = 10-20x cost reduction
5. **Batch Processing & Caching** (10% of value)
 - Batch API for 50% cost reduction
 - Prompt prefix caching (90%+ savings on repeated prompts)
 - Response caching for identical queries
 - Why: Production systems process millions of requests; optimization compounds

Explicitly Excluded (Low ROI / Interview Anti-Signals):

- ❌ Manual token counting ("just estimate")
 - *Why excluded:* Inaccurate, not scalable, signals amateur
- ❌ "Just use GPT-4 for everything"
 - *Why excluded:* 20x cost with 10% quality gain; not cost-aware
- ❌ Ignoring tokenization differences between models
 - *Why excluded:* Causes production bugs when switching models
- ❌ No cost monitoring ("we'll worry about that later")
 - *Why excluded:* Recipe for disaster; costs can 10x overnight
- ❌ Treating all tokens equally
 - *Why excluded:* Input tokens $\neq$ output tokens in cost; cached $\neq$ uncached

Hiring Signal Justification

What Tier 1 interviewers look for:

- Can you calculate exact cost for an LLM call before making it?
- Do you understand tokenizer differences between models?
- Can you optimize a prompt for cost without sacrificing quality?
- Have you implemented prompt caching or batch processing?
- Do you monitor token usage in production?

What signals "student not engineer":

- "I just send requests and hope it doesn't cost too much"
 - No understanding of how tokenization works
 - Treating context window as infinite
 - No cost attribution or monitoring
 - Using GPT-4 for tasks GPT-3.5 can handle
-

2. Genius-Level Understanding

Core Mental Models

Perspective 1: Infrastructure Engineer

Tokens are the currency of LLM systems. Track every penny or go bankrupt.

Traditional APIs:

```
API call cost = flat fee per request
Predictable: 1M requests = $X
```

LLM APIs:

```
API call cost = f(input_tokens, output_tokens, model, cache_hit)
Unpredictable: 1M requests = $100 to $100,000 depending on prompt design
```

Key insight: Token-level granularity means costs are highly variable and require active management.

Production implications:

python

```
# Amateur: No cost awareness
def process_document(doc):
    response = llm.complete(f"Summarize: {doc}")
    return response

# Cost for 1000 docs with avg 10k tokens each:
# Input: 1000 * 10k = 10M tokens
# Output: 1000 * 500 = 500k tokens
# GPT-4: (10M * $0.03 + 500k * $0.06) / 1000 = $330

# Engineer: Cost-aware with budget limits
class CostAwareLLM:
    def __init__(self, daily_budget=100):
        self.daily_budget = daily_budget
        self.daily_spend = 0
        self.reset_time = time.time()
```

```

def estimate_cost(self, prompt, max_tokens=500, model="gpt-4"):
    input_tokens = count_tokens(prompt, model)

    # Model pricing (per 1k tokens)
    pricing = {
        "gpt-4": {"input": 0.03, "output": 0.06},
        "gpt-3.5-turbo": {"input": 0.0015, "output": 0.002}
    }

    cost = (
        input_tokens / 1000 * pricing[model]["input"] +
        max_tokens / 1000 * pricing[model]["output"]
    )

    return cost

def complete(self, prompt, model="gpt-4", max_tokens=500):
    # Reset daily counter
    if time.time() - self.reset_time > 86400:
        self.daily_spend = 0
        self.reset_time = time.time()

    # Estimate cost
    estimated_cost = self.estimate_cost(prompt, max_tokens, model)

    # Check budget
    if self.daily_spend + estimated_cost > self.daily_budget:
        # Fallback to cheaper model
        if model == "gpt-4":
            logger.warning(f"Budget limit approaching, using GPT-3.5")
            model = "gpt-3.5-turbo"
            estimated_cost = self.estimate_cost(prompt, max_tokens, model)
        else:
            raise BudgetExceededError(
                f"Daily budget ${self.daily_budget} exceeded"
            )

    # Make request
    response = openai.ChatCompletion.create(
        model=model,
        messages=[{"role": "user", "content": prompt}],
        max_tokens=max_tokens
    )

    # Track actual cost
    actual_cost = (

```

```

        response.usage.prompt_tokens / 1000 * pricing[model]["input"] +
        response.usage.completion_tokens / 1000 * pricing[model]["output"]
    )

    self.daily_spend += actual_cost

    logger.info(
        f"Request cost: ${actual_cost:.4f}, "
        f"Daily total: ${self.daily_spend:.2f}/{self.daily_budget}"
    )

    return response.choices[0].message.content

```

Real example: Runaway costs

Startup scenario:

- Launch chatbot with GPT-4
- 1000 users × 50 messages/day
- Avg 500 input tokens, 300 output tokens per message

Daily cost:

1000 users × 50 msgs × (500 × \$0.03/1k + 300 × \$0.06/1k) = \$2,650/day

Monthly: \$79,500

After optimization (GPT-3.5 for 80% of queries):

- 80% GPT-3.5: 40k msgs × \$0.0035 = \$140
- 20% GPT-4: 10k msgs × \$0.03 = \$300

Daily: \$440

Monthly: \$13,200

Savings: \$66,300/month (83% reduction)

Perspective 2: Product Engineer

Context windows are finite budgets. Allocate tokens like you allocate money.

The Context Budget Allocation Problem:

python

```

# Given: 8k token context window
# Need to fit:
SYSTEM_PROMPT = 200 tokens      # Instructions
FEW_SHOT_EXAMPLES = 800 tokens  # Learning examples

```

```
CONVERSATION_HISTORY = ??? tokens # Previous turns
USER_MESSAGE = ??? tokens          # Current query
RETRIEVED_CONTEXT = ??? tokens     # RAG results

# Total must be ≤ 8000 tokens
```

Naive approach (fails):

python

```
def build_prompt(user_msg, history, rag_results):
    prompt = SYSTEM_PROMPT
    prompt += FEW_SHOT_EXAMPLES
    prompt += "\n".join(history) # All history
    prompt += "\n".join(rag_results) # All results
    prompt += user_msg

    return prompt # Often exceeds 8k tokens → API error
```

Engineer approach (priority-based allocation):

python

```
class ContextWindowManager:
    def __init__(self, max_tokens=8000, model="gpt-3.5-turbo"):
        self.max_tokens = max_tokens
        self.model = model
        self.encoder = tiktoken.encoding_for_model(model)

    def count_tokens(self, text):
        return len(self.encoder.encode(text))

    def build_prompt(self, user_msg, history, rag_results):
        # Priority-based allocation
        budget = self.max_tokens
        components = []

        # 1. System prompt (highest priority - always include)
        system_tokens = self.count_tokens(SYSTEM_PROMPT)
        components.append(("system", SYSTEM_PROMPT, system_tokens))
        budget -= system_tokens

        # 2. User message (second priority)
        user_tokens = self.count_tokens(user_msg)
```

```

components.append(("user", user_msg, user_tokens))
budget -= user_tokens

# 3. Reserve tokens for response
response_budget = 500
budget -= response_budget

# 4. Few-shot examples (third priority)
example_tokens = self.count_tokens(FEW_SHOT_EXAMPLES)
if budget >= example_tokens:
    components.append(("examples", FEW_SHOT_EXAMPLES, example_tokens))
    budget -= example_tokens
else:
    # Select most relevant examples that fit
    selected_examples = self.select_examples(
        user_msg, FEW_SHOT_EXAMPLES, budget
    )
    components.append(("examples", selected_examples, budget))
    budget = 0

# 5. RAG context (fourth priority)
rag_budget = min(budget * 0.6, 2000) # Max 2k for RAG
rag_context = self.truncate_rag(rag_results, rag_budget)
rag_tokens = self.count_tokens(rag_context)
components.append(("rag", rag_context, rag_tokens))
budget -= rag_tokens

# 6. Conversation history (lowest priority - fits what remains)
history_context = self.truncate_history(history, budget)
history_tokens = self.count_tokens(history_context)
components.append(("history", history_context, history_tokens))

# Build final prompt
prompt = "\n\n".join([c[1] for c in components])

# Verify we're within budget
total_tokens = sum(c[2] for c in components)
assert total_tokens <= self.max_tokens, "Prompt exceeds context window"

# Log token allocation
logger.info(f"Token allocation: {total_tokens}/{self.max_tokens}")
for name, _, tokens in components:
    logger.info(f"  {name}: {tokens} tokens")

return prompt

```

```

def truncate_history(self, history, budget):
    """Keep most recent messages that fit in budget"""
    truncated = []
    tokens_used = 0

    # Iterate from most recent
    for msg in reversed(history):
        msg_tokens = self.count_tokens(msg)
        if tokens_used + msg_tokens <= budget:
            truncated.insert(0, msg)
            tokens_used += msg_tokens
        else:
            break

    return "\n".join(truncated)

def truncate_rag(self, results, budget):
    """Keep top results that fit in budget"""
    truncated = []
    tokens_used = 0

    for result in results: # Assumed sorted by relevance
        result_tokens = self.count_tokens(result)
        if tokens_used + result_tokens <= budget:
            truncated.append(result)
            tokens_used += result_tokens
        else:
            break

    return "\n---\n".join(truncated)

```

Why this matters:

- No API errors from exceeding context window
- Predictable token usage
- Graceful degradation (less history, not failure)
- Observable (logs show token allocation)

Perspective 3: Researcher

Tokenization is lossy compression of text into subword units.

Advanced insight: Different tokenizers fragment text differently

python


```

# Example text
text = "GPT-4 tokenizes differently than GPT-3.5-turbo"

# OpenAI (BPE tokenizer)
import tiktoken
enc_gpt4 = tiktoken.encoding_for_model("gpt-4")
enc_gpt35 = tiktoken.encoding_for_model("gpt-3.5-turbo")

print(f"GPT-4 tokens: {len(enc_gpt4.encode(text))}")
# Output: 11 tokens

print(f"GPT-3.5 tokens: {len(enc_gpt35.encode(text))}")
# Output: 12 tokens

# Actual tokens
print(enc_gpt4.encode(text))
# [38, 2898, 12, 19, 1492, 4861, 22009, 1109, 480, 2899, 12, 18, 13, 20, 12, 66, 39305]

# Note: Different token IDs between models!

```

Why this matters in production:

python

```

# Scenario: Switch from GPT-3.5 to GPT-4
# Old code (wrong):
MAX_TOKENS = 4000 # Based on GPT-3.5 testing

# Switches to GPT-4
# Problem: Same prompt might be 10-15% more tokens
# Result: Context window overflow errors

# Correct approach: Model-specific token counting
def count_tokens(text, model):
    encoder = tiktoken.encoding_for_model(model)
    return len(encoder.encode(text))

# Dynamic max tokens based on model
def get_max_tokens(model):
    limits = {
        "gpt-4": 8192,
        "gpt-4-32k": 32768,
        "gpt-3.5-turbo": 4096,
        "gpt-3.5-turbo-16k": 16384
    }

```

```
}  
return limits.get(model, 4096)
```

Counterintuitive tokenization behaviors:

python

```
# Trailing spaces matter  
text1 = "Hello world"  
text2 = "Hello world " # Note trailing space  
  
print(len(enc.encode(text1))) # 2 tokens  
print(len(enc.encode(text2))) # 3 tokens (space is a token!)  
  
# Repeated characters compress well  
text1 = "a" * 100  
text2 = "abcdefgh" * 12 # Same length  
  
print(len(enc.encode(text1))) # 25 tokens (compression via repetition)  
print(len(enc.encode(text2))) # 60 tokens (no compression)  
  
# Code compresses worse than prose  
prose = "The quick brown fox jumps over the lazy dog." * 10  
code = "def func():\n    return True\n" * 10  
  
print(f"Prose: {len(enc.encode(prose))} tokens") # ~90 tokens  
print(f"Code: {len(enc.encode(code))} tokens") # ~140 tokens  
# Code has more special chars → more tokens per character
```

Perspective 4: Hiring Manager

Show me you understand the economics of LLM operations.

Red flags:

- "I don't track token usage" (unacceptable in production)
- "I always use max_tokens=4000" (wastes money)
- Can't explain how tokenization works
- No cost optimization strategy
- Treating all models as equivalent (GPT-4 vs GPT-3.5)

Green flags:

- Real-time cost dashboard in portfolio

- Per-feature cost breakdown
 - Token-level optimization (50%+ reduction documented)
 - A/B tests showing cost/quality tradeoffs
 - Batch processing or caching implementation
-

Advanced Production Use Cases

Case 1: Dynamic Few-Shot Selection Based on Token Budget

Problem: Static few-shot examples waste tokens when input is large.

Solution: Dynamically select examples based on remaining budget.

python

```
class DynamicFewShotSelector:
    def __init__(self, example_bank, encoder):
        self.example_bank = example_bank
        self.encoder = encoder

        # Pre-compute example embeddings for similarity search
        from sentence_transformers import SentenceTransformer
        self.embedding_model = SentenceTransformer('all-MiniLM-L6-v2')
        self.example_embeddings = self.embedding_model.encode(
            [ex['input'] for ex in example_bank]
        )

    def select_examples(self, query, token_budget):
        """Select most relevant examples that fit in budget"""
        # Find similar examples
        query_emb = self.embedding_model.encode([query])
        similarities = cosine_similarity(query_emb, self.example_embeddings)[0]

        # Sort by similarity
        ranked_indices = np.argsort(similarities[::-1])

        # Select examples that fit in budget
        selected = []
        tokens_used = 0

        for idx in ranked_indices:
            example = self.example_bank[idx]
```

```

        example_text = f"Input: {example['input']}\nOutput: {example['output']}\n"
        example_tokens = len(self.encoder.encode(example_text))

        if tokens_used + example_tokens <= token_budget:
            selected.append(example_text)
            tokens_used += example_tokens
        else:
            break

    return "\n".join(selected), tokens_used

# Usage
selector = DynamicFewShotSelector(EXAMPLE_BANK, encoder)

# Large input → fewer examples
large_input = "..." * 1000 # 3000 tokens
examples, tokens = selector.select_examples(large_input, token_budget=500)
# Result: 2 most relevant examples (400 tokens)

# Small input → more examples
small_input = "How do I install Python?" # 10 tokens
examples, tokens = selector.select_examples(small_input, token_budget=1500)
# Result: 8 examples (1400 tokens)

```

Impact: 30-40% token savings while maintaining quality (most relevant examples preserved).

Case 2: Prompt Caching for Repeated System Instructions

Problem: System prompts repeated on every request waste money.

Solution: Use prompt caching (where available) or implement application-level caching.

python

```

# Anthropic Claude example (native prompt caching)
import anthropic

client = anthropic.Anthropic()

# Mark cacheable content
message = client.messages.create(
    model="claude-3-5-sonnet-20241022",
    max_tokens=1024,
    system=[
        {

```

```

        "type": "text",
        "text": "You are an expert Python tutor..." # Long system prompt
    },
    {
        "type": "text",
        "text": REFERENCE_DOCS, # 10k tokens of documentation
        "cache_control": {"type": "ephemeral"} # Cache this!
    }
],
messages=[
    {"role": "user", "content": "How do I use decorators?"}
]
)

# First request:
# - Input tokens: 11,000 (system + docs + user)
# - Cost: $0.33 (11k × $0.03/1k)

# Second request (same system prompt):
# - Input tokens: 1,000 (only user message, rest cached)
# - Cached tokens: 10,000
# - Cost: $0.03 (1k × $0.03/1k) + $0.003 (10k × $0.0003/1k cache read)
# - Total: $0.033

# Savings: $0.33 → $0.033 = 90% reduction

```

Application-level caching (for models without native caching):

python

```

import hashlib
import json

class PromptCacheManager:
    def __init__(self, cache_ttl=3600):
        self.cache = {}
        self.cache_ttl = cache_ttl

    def _hash_prompt(self, prompt):
        """Create deterministic hash of prompt"""
        return hashlib.sha256(
            json.dumps(prompt, sort_keys=True).encode()
        ).hexdigest()

    def get_cached_response(self, prompt, temperature=0):

```

```

        """Get cached response if available"""
        # Only cache deterministic responses
        if temperature > 0:
            return None

        prompt_hash = self._hash_prompt(prompt)

        if prompt_hash in self.cache:
            cached_item = self.cache[prompt_hash]

            # Check if cache is still valid
            if time.time() - cached_item['timestamp'] < self.cache_ttl:
                logger.info(f"Cache hit for prompt hash {prompt_hash[:8]}")
                return cached_item['response']
            else:
                # Expired
                del self.cache[prompt_hash]

        return None

    def cache_response(self, prompt, response, temperature=0):
        """Cache response"""
        if temperature > 0:
            return # Don't cache non-deterministic

        prompt_hash = self._hash_prompt(prompt)
        self.cache[prompt_hash] = {
            'response': response,
            'timestamp': time.time()
        }

    def get_stats(self):
        return {
            'cache_size': len(self.cache),
            'cache_entries': list(self.cache.keys())[:5]
        }

# Usage with OpenAI
cache = PromptCacheManager()

def cached_complete(prompt, temperature=0):
    # Check cache
    cached = cache.get_cached_response(prompt, temperature)
    if cached:
        return cached

```

```

# Call API
response = openai.ChatCompletion.create(
    model="gpt-3.5-turbo",
    messages=prompt,
    temperature=temperature
)

result = response.choices[0].message.content

# Cache result
cache.cache_response(prompt, result, temperature)

return result

# Test
prompt = [{"role": "user", "content": "What is Python?"}]

# First call: API request ($$$)
result1 = cached_complete(prompt)

# Second call: Cached (free!)
result2 = cached_complete(prompt)

assert result1 == result2
print(cache.get_stats())
# {'cache_size': 1, ...}

```

Impact: 60-90% cost reduction for systems with repeated queries.

Case 3: Intelligent Model Routing Based on Complexity

Problem: Using GPT-4 for simple tasks wastes money.

Solution: Route based on query complexity.

python

```

class IntelligentRouter:
    def __init__(self):
        self.complexity_classifier = self._load_classifier()

    # Model costs (per 1k tokens)
    self.costs = {
        "gpt-4": {"input": 0.03, "output": 0.06},
        "gpt-3.5-turbo": {"input": 0.0015, "output": 0.002}
    }

```

```

    }

def _load_classifier(self):
    """Train classifier on query complexity"""
    # Simple heuristic-based classifier
    # Production: Use ML model trained on labeled data
    return None

def estimate_complexity(self, query):
    """Estimate query complexity (0-1)"""
    # Heuristics:
    factors = []

    # 1. Length (longer = more complex)
    factors.append(min(len(query.split()) / 100, 1.0))

    # 2. Technical terms
    technical_words = ["implement", "architecture", "algorithm", "optimize"]
    tech_score = sum(1 for word in technical_words if word in query.lower())
    factors.append(min(tech_score / 3, 1.0))

    # 3. Multi-step reasoning indicators
    reasoning_indicators = ["first", "then", "because", "therefore"]
    reasoning_score = sum(1 for ind in reasoning_indicators if ind in query.lower())
    factors.append(min(reasoning_score / 2, 1.0))

    # 4. Question complexity
    if "?" in query:
        question_marks = query.count("?")
        factors.append(min(question_marks / 3, 1.0))

    return np.mean(factors)

def route_query(self, query, quality_threshold=0.5):
    """Route to appropriate model"""
    complexity = self.estimate_complexity(query)

    if complexity > quality_threshold:
        model = "gpt-4"
        reason = f"High complexity ({complexity:.2f})"
    else:
        model = "gpt-3.5-turbo"
        reason = f"Low complexity ({complexity:.2f})"

    logger.info(f"Routing to {model}: {reason}")
    return model

```



```

def complete_with_routing(self, query):
    """Complete with automatic routing"""
    model = self.route_query(query)

    response = openai.ChatCompletion.create(
        model=model,
        messages=[{"role": "user", "content": query}]
    )

    # Track cost
    usage = response.usage
    cost = (
        usage.prompt_tokens / 1000 * self.costs[model]["input"] +
        usage.completion_tokens / 1000 * self.costs[model]["output"]
    )

    logger.info(f"Request cost: ${cost:.4f} (model: {model})")

    return response.choices[0].message.content

# Usage
router = IntelligentRouter()

# Simple query → GPT-3.5 ($0.001)
router.complete_with_routing("What is Python?")

# Complex query → GPT-4 ($0.05)
router.complete_with_routing(
    "Explain the tradeoffs between microservices and monolithic "
    "architecture, then provide a decision framework for choosing "
    "between them based on team size, deployment frequency, and "
    "system complexity."
)

```

Impact: 70-85% cost reduction on mixed workloads (measured on real customer support data).

Expert-Level Comprehension Tests

Question 1: Your production system's LLM costs jumped from \$500/day to \$5,000/day overnight. No code changes were deployed. What are your top 3 diagnostic hypotheses and how would you investigate each?

Expected Answer

Hypotheses: 1. **Traffic spike or bot attack** (most likely) - Diagnosis: Check request volume metrics by endpoint/user - Look for: Unusual request patterns, single user making 1000s of requests - Investigation: python

```
# Analyze request logs
df = pd.read_csv('request_logs.csv')
print(df.groupby('user_id')['cost'].sum().sort_values(ascending=False).head(10))
# Look for outlier users
```

- Fix: Rate limiting, ban malicious users 2. **Prompt injection causing massive output** (second likely) - Diagnosis: Check average output tokens per request - Look for: Prompts generating 4000+ token responses - Investigation: python

```
# Check output token distribution
print(df['output_tokens'].describe())
print(df[df['output_tokens'] > 2000].sample(10))
# Review prompts with unusually long outputs
```

- Fix: Add max_tokens limits, stricter output validation 3. **Accidental model upgrade** (API provider changed default) - Diagnosis: Check which models are being called - Look for: Unexpected GPT-4 usage where GPT-3.5 was intended - Investigation: python

```
print(df.groupby('model')['cost'].sum())
# Compare to yesterday's distribution
```

- Fix: Pin model versions explicitly in code **Anti-pattern:** "Maybe the API got more expensive?" (Should check provider's changelog, but costs don't change overnight)

Question 2: You need to process 1M customer support tickets with an LLM. Each ticket averages 500 tokens. Using GPT-4 costs \$30k, GPT-3.5 costs \$1.5k. How would you achieve <\$5k total cost while maintaining >90% quality?

Expected Answer

Strategy: Hybrid approach with confidence-based routing python

```
class HybridProcessor:
    def __init__(self):
        self.gpt35_cost = 0.0035 # per request
        self.gpt4_cost = 0.03    # per request
        self.budget = 5000
```

```

def process_batch(self, tickets):
    results = []
    cost = 0

    for ticket in tickets:
        # Step 1: Try GPT-3.5 first
        response_35 = self.process_gpt35(ticket)
        cost += self.gpt35_cost

        # Step 2: Check confidence
        if response_35['confidence'] > 0.85:
            # High confidence → accept
            results.append(response_35)
        else:
            # Low confidence → escalate to GPT-4
            response_4 = self.process_gpt4(ticket)
            cost += self.gpt4_cost
            results.append(response_4)

        if cost > self.budget:
            raise BudgetExceededError()

    return results

# Expected distribution:
# - 80% high confidence (GPT-3.5 only): 800k × $0.0035 = $2,800
# - 20% low confidence (GPT-3.5 + GPT-4): 200k × ($0.0035 + $0.03) = $6,700
# Total: $9,500 (over budget!)

# Optimization: Use batch API for 50% discount
# - GPT-3.5 batch: $0.00175
# - GPT-4 batch: $0.015

# Revised:
# - 800k × $0.00175 = $1,400
# - 200k × ($0.00175 + $0.015) = $3,350
# Total: $4,750 ✓

```

Additional optimizations:

- Cache responses:** Many tickets are similar (30-40% cache hit rate)
- Rule-based pre-filtering:** Handle 20% with regex/rules (free)
- Prompt compression:** Reduce ticket to key facts only (-30% tokens)

Final cost breakdown:

- 20% rules: $200k \times \$0 = \0
- 50% GPT-3.5: $500k \times \$0.00175 = \875
- 30% GPT-4: $300k \times \$0.015 = \$4,500$ (with batch)

Total: \$5,375 (slightly over, negotiate or reduce GPT-4 to 25%)

****Quality assurance:**** - Eval on 1000 labeled tickets - Measure: accuracy, precision, recall - Target: >90% across all metrics

Question 3: A colleague suggests "just increasing max_tokens from 500 to 2000 to avoid truncation." What are the cost and latency implications? How would you respond?

Expected Answer

****Cost implications:**** python

```
# Scenario: 100k requests/day
requests_per_day = 100_000

# Current (max_tokens=500)
avg_output_tokens_current = 400 # Typical actual usage
cost_current = (
    requests_per_day * avg_output_tokens_current / 1000 * 0.002
)
print(f"Current daily cost: ${cost_current}")
# $80/day

# Proposed (max_tokens=2000)
# Problem: max_tokens affects API pricing even if not used!
# (For some providers like Anthropic)

# Also: Signals to model to generate longer responses
avg_output_tokens_proposed = 1200 # Model tends to fill budget
cost_proposed = (
    requests_per_day * avg_output_tokens_proposed / 1000 * 0.002
)
print(f"Proposed daily cost: ${cost_proposed}")
# $240/day

# Cost increase: 3x ($160/day extra = $4,800/month)
```

****Latency implications:****

```
Tokens/second generation: ~40-50 tokens/sec

Current (500 max):
- Avg 400 tokens → 8-10 seconds

Proposed (2000 max):
- Avg 1200 tokens → 24-30 seconds

User experience degradation:
```

- 8 sec: Acceptable
- 25 sec: User abandonment, timeout issues

Better alternatives: python

```
# 1. Dynamic max_tokens based on input
def calculate_max_tokens(input_text):
    input_tokens = count_tokens(input_text)

    # Rule: Output should be ~50-80% of input
    return min(int(input_tokens * 0.8), 1000)

# 2. Streaming with early stopping
def stream_with_stop(prompt):
    for chunk in openai.ChatCompletion.create(
        prompt=prompt,
        max_tokens=2000,
        stream=True
    ):
        text = chunk.choices[0].delta.content
        yield text

        # Stop if complete thought detected
        if is_complete_response(accumulated_text):
            break

# 3. Two-stage approach
def smart_completion(prompt):
    # Stage 1: Generate with reasonable limit
    response = complete(prompt, max_tokens=500)

    # Stage 2: If truncated, continue
    if is_truncated(response):
        continuation = complete(
            prompt + response + "\n[Continue]",
            max_tokens=500
        )
        return response + continuation

    return response
```

Response to colleague: "Increasing max_tokens 4x will: 1. Triple our costs (\$5k/month extra)
2. Double our latency (user experience hit) 3. Not solve truncation for truly long responses
Instead, let's: 1. Implement dynamic max_tokens based on input 2. Add streaming with early

stopping 3. Analyze what % of responses are actually truncated (probably <5%) 4. For those cases, use continuation prompts"

3. Fastest Path to Competence (Execution-Focused)

Prerequisites (Week 0: Setup)

Install:

bash

```
pip install \
  tiktoken \           # OpenAI tokenizer
  openai anthropic \   # LLM APIs
  pandas matplotlib \  # Analytics
  python-dotenv \      # Config management
  pytest \             # Testing
  redis                 # Optional: Response caching
```

Get API access:

- OpenAI: \$5 free credits (new accounts) or \$10-20 for curriculum
- Anthropic: Check for credits or use OpenRouter as alternative
- Set up billing alerts at \$5, \$10, \$20 thresholds

Repository structure:

```
token-optimization/
├── src/
│   ├── token_counter.py      # Model-specific counting
│   ├── cost_tracker.py       # Real-time cost monitoring
│   ├── context_manager.py    # Budget allocation
│   ├── prompt_compressor.py  # Optimization techniques
│   └── model_router.py        # Intelligent routing
├── data/
│   ├── cost_logs.csv         # Request-level costs
│   └── optimization_results.json
├── notebooks/
│   └── cost_analysis.ipynb    # Visualizations
└── tests/
```

```
| └─ test_optimizations.py
└─ README.md
```

Week 1: Token Counting & Cost Tracking

Learning Objectives:

- Implement accurate token counting for multiple models
- Build real-time cost tracking system
- Create cost attribution framework

Day 1-2: Model-Specific Token Counting

Hands-on Lab:

python

```
# src/token_counter.py
import tiktoken
from typing import Dict, List, Union

class TokenCounter:
    def __init__(self):
        # Cache encoders
        self.encoders = {}

    def _get_encoder(self, model: str):
        """Get or create encoder for model"""
        if model not in self.encoders:
            try:
                self.encoders[model] = tiktoken.encoding_for_model(model)
            except KeyError:
                # Fallback to cl100k_base for unknown models
                self.encoders[model] = tiktoken.get_encoding("cl100k_base")

        return self.encoders[model]

    def count_tokens(self, text: Union[str, List[Dict]], model: str = "gpt-3.5-turbo") -> int:
        """Count tokens for text or message list"""
        encoder = self._get_encoder(model)

        if isinstance(text, str):
```

```

        return len(encoder.encode(text))

    elif isinstance(text, list):
        # Chat messages format
        return self._count_message_tokens(text, model)

    else:
        raise ValueError(f"Unsupported text type: {type(text)}")

def _count_message_tokens(self, messages: List[Dict], model: str) -> int:
    """Count tokens in chat messages (accounts for special tokens)"""
    encoder = self._get_encoder(model)

    # Token counting varies by model
    if model in ["gpt-3.5-turbo", "gpt-4"]:
        tokens_per_message = 4 # <im_start>, role, content, <im_end>
        tokens_per_name = 1
    else:
        tokens_per_message = 3
        tokens_per_name = 1

    num_tokens = 0
    for message in messages:
        num_tokens += tokens_per_message
        for key, value in message.items():
            num_tokens += len(encoder.encode(value))
            if key == "name":
                num_tokens += tokens_per_name

    num_tokens += 2 # Assistant's reply priming
    return num_tokens

def estimate_cost(self,
                  input_text: Union[str, List[Dict]],
                  output_tokens: int,
                  model: str = "gpt-3.5-turbo") -> float:
    """Estimate cost for a request"""
    # Pricing per 1k tokens
    pricing = {
        "gpt-4": {"input": 0.03, "output": 0.06},
        "gpt-4-turbo": {"input": 0.01, "output": 0.03},
        "gpt-3.5-turbo": {"input": 0.0015, "output": 0.002},
        "claude-3-opus": {"input": 0.015, "output": 0.075},
        "claude-3-sonnet": {"input": 0.003, "output": 0.015}
    }

```



```

        if model not in pricing:
            # Default to GPT-3.5 pricing
            model_pricing = pricing["gpt-3.5-turbo"]
        else:
            model_pricing = pricing[model]

        input_tokens = self.count_tokens(input_text, model)

        cost = (
            input_tokens / 1000 * model_pricing["input"] +
            output_tokens / 1000 * model_pricing["output"]
        )

        return cost

def analyze_prompt(self, prompt: Union[str, List[Dict]], model: str = "gpt-3.5-turbo")
    """Detailed prompt analysis"""
    if isinstance(prompt, str):
        messages = [{"role": "user", "content": prompt}]
    else:
        messages = prompt

    encoder = self._get_encoder(model)

    analysis = {
        "total_tokens": self.count_tokens(messages, model),
        "total_chars": sum(len(m.get("content", "")) for m in messages),
        "messages": []
    }

    for msg in messages:
        content = msg.get("content", "")
        msg_tokens = len(encoder.encode(content))

        analysis["messages"].append({
            "role": msg.get("role"),
            "tokens": msg_tokens,
            "chars": len(content),
            "tokens_per_char": msg_tokens / len(content) if content else 0
        })

    return analysis

# Usage examples
counter = TokenCounter()

```

```

# Simple text
text = "Hello, how are you?"
tokens = counter.count_tokens(text, "gpt-3.5-turbo")
print(f"Tokens: {tokens}") # 6

# Chat messages
messages = [
    {"role": "system", "content": "You are a helpful assistant."},
    {"role": "user", "content": "What is Python?"}
]
tokens = counter.count_tokens(messages, "gpt-4")
print(f"Message tokens: {tokens}") # ~20

# Cost estimation
cost = counter.estimate_cost(messages, output_tokens=100, model="gpt-4")
print(f"Estimated cost: ${cost:.4f}") # ~$0.0066

# Detailed analysis
analysis = counter.analyze_prompt(messages, "gpt-4")
print(json.dumps(analysis, indent=2))

```

Deliverable: Token counter that:

- Handles multiple models correctly
- Accounts for special tokens
- Provides detailed breakdowns

Resources:

- [tiktoken GitHub](#)
- [OpenAI Tokenizer](#)

Day 3-5: Real-Time Cost Tracking

Hands-on Lab:

python

```

# src/cost_tracker.py
import time
import pandas as pd
from pathlib import Path
from datetime import datetime

class CostTracker:

```

```

def __init__(self, log_file="data/cost_logs.csv"):
    self.log_file = Path(log_file)
    self.log_file.parent.mkdir(exist_ok=True)

    # Initialize log file if doesn't exist
    if not self.log_file.exists():
        pd.DataFrame(columns=[
            'timestamp', 'model', 'input_tokens', 'output_tokens',
            'cost', 'request_id', 'endpoint', 'user_id'
        ]).to_csv(self.log_file, index=False)

    self.session_start = time.time()
    self.session_cost = 0

def log_request(self,
                model: str,
                input_tokens: int,
                output_tokens: int,
                cost: float,
                request_id: str = None,
                endpoint: str = None,
                user_id: str = None):
    """Log a single request"""
    log_entry = {
        'timestamp': datetime.now().isoformat(),
        'model': model,
        'input_tokens': input_tokens,
        'output_tokens': output_tokens,
        'cost': cost,
        'request_id': request_id or f"req_{int(time.time()*1000)}",
        'endpoint': endpoint,
        'user_id': user_id
    }

    # Append to CSV
    pd.DataFrame([log_entry]).to_csv(
        self.log_file,
        mode='a',
        header=False,
        index=False
    )

    self.session_cost += cost

    return log_entry

```

```

def get_stats(self,
               start_time: datetime = None,
               end_time: datetime = None,
               group_by: str = None):
    """Get cost statistics"""
    df = pd.read_csv(self.log_file)
    df['timestamp'] = pd.to_datetime(df['timestamp'])

    # Filter by time range
    if start_time:
        df = df[df['timestamp'] >= start_time]
    if end_time:
        df = df[df['timestamp'] <= end_time]

    stats = {
        'total_requests': len(df),
        'total_cost': df['cost'].sum(),
        'total_input_tokens': df['input_tokens'].sum(),
        'total_output_tokens': df['output_tokens'].sum(),
        'avg_cost_per_request': df['cost'].mean(),
        'avg_tokens_per_request': (df['input_tokens'] + df['output_tokens']).mean()
    }

    # Group by if specified
    if group_by and group_by in df.columns:
        grouped = df.groupby(group_by).agg({
            'cost': ['sum', 'mean', 'count'],
            'input_tokens': 'sum',
            'output_tokens': 'sum'
        }).round(4)
        stats['breakdown'] = grouped.to_dict()

    return stats

def get_session_stats(self):
    """Get current session statistics"""
    duration = time.time() - self.session_start

    return {
        'session_duration_minutes': duration / 60,
        'session_cost': self.session_cost,
        'cost_per_minute': self.session_cost / (duration / 60) if duration > 0 else 0
    }

def check_budget(self, budget_limit: float, time_window_hours: int = 24):
    """Check if within budget"""

```

```

start_time = datetime.now() - pd.Timedelta(hours=time_window_hours)
stats = self.get_stats(start_time=start_time)

current_spend = stats['total_cost']
remaining = budget_limit - current_spend

return {
    'budget_limit': budget_limit,
    'current_spend': current_spend,
    'remaining': remaining,
    'utilization': current_spend / budget_limit if budget_limit > 0 else 0,
    'within_budget': current_spend <= budget_limit
}

def generate_report(self, output_file="cost_report.txt"):
    """Generate cost report"""
    # Last 24 hours stats
    start_time = datetime.now() - pd.Timedelta(hours=24)
    stats = self.get_stats(start_time=start_time, group_by='model')

    report = [
        "=" * 60,
        "LLM COST REPORT - Last 24 Hours",
        "=" * 60,
        f"\nTotal Requests: {stats['total_requests']:,}",
        f"Total Cost: ${stats['total_cost']:.2f}",
        f"Avg Cost/Request: ${stats['avg_cost_per_request']:.4f}",
        f"\nTotal Input Tokens: {stats['total_input_tokens']:,}",
        f"Total Output Tokens: {stats['total_output_tokens']:,}",
        f"Avg Tokens/Request: {stats['avg_tokens_per_request']:.0f}",
        "\n" + "=" * 60,
        "BREAKDOWN BY MODEL",
        "=" * 60
    ]

    if 'breakdown' in stats:
        for model, data in stats['breakdown']['cost']['sum'].items():
            cost = data
            count = stats['breakdown']['cost']['count'][model]
            avg = stats['breakdown']['cost']['mean'][model]

            report.append(f"\n{model}:")
            report.append(f"  Requests: {count:,}")
            report.append(f"  Total Cost: ${cost:.2f}")
            report.append(f"  Avg Cost: ${avg:.4f}")

```

```

        report_text = "\n".join(report)

        # Write to file
        with open(output_file, 'w') as f:
            f.write(report_text)

        print(report_text)
        return report_text

# Usage
tracker = CostTracker()

# Simulate requests
tracker.log_request(
    model="gpt-3.5-turbo",
    input_tokens=500,
    output_tokens=200,
    cost=0.00135,
    endpoint="/api/chat",
    user_id="user123"
)

tracker.log_request(
    model="gpt-4",
    input_tokens=800,
    output_tokens=400,
    cost=0.048,
    endpoint="/api/chat",
    user_id="user456"
)

# Get stats
stats = tracker.get_stats(group_by='model')
print(json.dumps(stats, indent=2))

# Check budget
budget_status = tracker.check_budget(budget_limit=100.0, time_window_hours=24)
print(budget_status)

# Generate report
tracker.generate_report()

```

Deliverable: Cost tracking system with:

- Request-level logging

- Real-time budget monitoring
- Automated reporting

Day 6-7: Cost Attribution & Visualization

Hands-on Lab:

python

```
# notebooks/cost_analysis.ipynb
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

class CostAnalyzer:
    def __init__(self, log_file="data/cost_logs.csv"):
        self.df = pd.read_csv(log_file)
        self.df['timestamp'] = pd.to_datetime(self.df['timestamp'])
        self.df['hour'] = self.df['timestamp'].dt.hour
        self.df['date'] = self.df['timestamp'].dt.date

    def plot_cost_over_time(self):
        """Cost trend over time"""
        daily_cost = self.df.groupby('date')['cost'].sum()

        plt.figure(figsize=(12, 6))
        plt.plot(daily_cost.index, daily_cost.values, marker='o')
        plt.title('Daily LLM Costs')
        plt.xlabel('Date')
        plt.ylabel('Cost ($)')
        plt.xticks(rotation=45)
        plt.grid(True, alpha=0.3)
        plt.tight_layout()
        plt.savefig('daily_costs.png')
        plt.show()

    def plot_model_distribution(self):
        """Cost distribution by model"""
        model_costs = self.df.groupby('model')['cost'].sum().sort_values(ascending=False)

        plt.figure(figsize=(10, 6))
        model_costs.plot(kind='bar')
        plt.title('Cost by Model')
        plt.xlabel('Model')
        plt.ylabel('Total Cost ($)')
```

```

plt.xticks(rotation=45)
plt.tight_layout()
plt.savefig('model_distribution.png')
plt.show()

def plot_hourly_pattern(self):
    """Usage pattern by hour"""
    hourly = self.df.groupby('hour').agg({
        'cost': 'sum',
        'request_id': 'count'
    })

    fig, ax1 = plt.subplots(figsize=(12, 6))

    ax1.bar(hourly.index, hourly['request_id'], alpha=0.7, label='Requests')
    ax1.set_xlabel('Hour of Day')
    ax1.set_ylabel('Number of Requests', color='blue')
    ax1.tick_params(axis='y', labelcolor='blue')

    ax2 = ax1.twinx()
    ax2.plot(hourly.index, hourly['cost'], color='red', marker='o', label='Cost')
    ax2.set_ylabel('Cost ($)', color='red')
    ax2.tick_params(axis='y', labelcolor='red')

    plt.title('Hourly Usage Pattern')
    plt.tight_layout()
    plt.savefig('hourly_pattern.png')
    plt.show()

def identify_cost_outliers(self, threshold_percentile=95):
    """Find expensive requests"""
    threshold = self.df['cost'].quantile(threshold_percentile / 100)
    outliers = self.df[self.df['cost'] > threshold].sort_values('cost', ascending=False)

    print(f"\nCost Outliers (>{threshold_percentile}th percentile = ${threshold:.4f})")
    print(outliers[['timestamp', 'model', 'input_tokens', 'output_tokens', 'cost', 'request_id']])

    return outliers

def generate_insights(self):
    """Generate actionable insights"""
    insights = []

    # 1. Model efficiency
    model_efficiency = self.df.groupby('model').agg({
        'cost': 'mean',

```



```

        'input_tokens': 'mean',
        'output_tokens': 'mean'
    })

    insights.append("Model Efficiency:")
    for model, row in model_efficiency.iterrows():
        insights.append(f"  {model}: ${row['cost']:.4f}/request "
                        f"({row['input_tokens']:.0f} in, {row['output_tokens']:.0f} out)")

    # 2. Cost concentration
    if 'user_id' in self.df.columns:
        user_costs = self.df.groupby('user_id')['cost'].sum().sort_values(ascending=True)
        top_10_pct = user_costs.head(int(len(user_costs) * 0.1)).sum() / user_costs.sum()
        insights.append(f"\nTop 10% users account for {top_10_pct:.1%} of costs")

    # 3. Optimization opportunities
    gpt4_usage = self.df[self.df['model'] == 'gpt-4']['cost'].sum()
    total_cost = self.df['cost'].sum()
    if gpt4_usage / total_cost > 0.5:
        potential_savings = gpt4_usage * 0.5 # If 50% could use GPT-3.5
        insights.append(f"\nPotential savings: ${potential_savings:.2f} "
                        "by routing simple queries to GPT-3.5")

    return "\n".join(insights)

# Usage
analyzer = CostAnalyzer()
analyzer.plot_cost_over_time()
analyzer.plot_model_distribution()
analyzer.plot_hourly_pattern()
outliers = analyzer.identify_cost_outliers()
print(analyzer.generate_insights())

```

Deliverable: Analytics dashboard showing:

- Daily/hourly cost trends
 - Per-model cost breakdown
 - Cost outlier detection
 - Optimization opportunities
-

Week 2: Context Window Management & Compression

Learning Objectives:

- Implement priority-based context allocation
- Build prompt compression techniques
- Optimize token usage without quality loss

Day 8-10: Context Window Manager

Hands-on Lab:

python

```
# src/context_manager.py
from token_counter import TokenCounter

class ContextWindowManager:
    def __init__(self, model="gpt-3.5-turbo", max_tokens=4096):
        self.model = model
        self.max_tokens = max_tokens
        self.counter = TokenCounter()

    def allocate_budget(self, components: dict, response_tokens: int = 500):
        """
        Allocate token budget across prompt components

        components = {
            'system': {'text': '...', 'priority': 10}, # Highest priority
            'examples': {'text': '...', 'priority': 7},
            'context': {'text': '...', 'priority': 5},
            'history': {'text': '...', 'priority': 3},
            'user': {'text': '...', 'priority': 10} # Highest priority
        }
        """
        # Reserve tokens for response
        available_tokens = self.max_tokens - response_tokens

        # Calculate tokens for each component
        for name, comp in components.items():
            comp['tokens'] = self.counter.count_tokens(comp['text'], self.model)

        # Sort by priority (highest first)
        sorted_components = sorted(
            components.items(),
```

```

        key=lambda x: x[1]['priority'],
        reverse=True
    )

    # Allocate tokens
    allocated = {}
    remaining_budget = available_tokens

    for name, comp in sorted_components:
        if comp['tokens'] <= remaining_budget:
            # Fits entirely
            allocated[name] = comp['text']
            remaining_budget -= comp['tokens']
        elif remaining_budget > 0:
            # Truncate to fit
            truncated = self.truncate_to_budget(
                comp['text'],
                remaining_budget,
                strategy=comp.get('truncation', 'tail')
            )
            allocated[name] = truncated
            remaining_budget = 0
        else:
            # No budget left
            allocated[name] = ""

    # Build final prompt
    prompt_parts = [allocated[name] for name, _ in sorted_components if allocated[name]]
    final_prompt = "\n\n".join(prompt_parts)

    # Verify within budget
    final_tokens = self.counter.count_tokens(final_prompt, self.model)
    assert final_tokens <= available_tokens, "Prompt exceeds budget"

    return {
        'prompt': final_prompt,
        'tokens_used': final_tokens,
        'tokens_available': available_tokens,
        'utilization': final_tokens / available_tokens
    }

def truncate_to_budget(self, text: str, budget: int, strategy: str = 'tail'):
    """Truncate text to fit token budget"""
    current_tokens = self.counter.count_tokens(text, self.model)

    if current_tokens <= budget:

```

```

        return text

    if strategy == 'head':
        # Keep beginning
        return self._truncate_head(text, budget)
    elif strategy == 'tail':
        # Keep end
        return self._truncate_tail(text, budget)
    elif strategy == 'middle':
        # Keep beginning and end
        return self._truncate_middle(text, budget)
    else:
        raise ValueError(f"Unknown strategy: {strategy}")

    def _truncate_head(self, text: str, budget: int):
        """Keep first N tokens"""
        tokens = self.counter._get_encoder(self.model).encode(text)
        truncated_tokens = tokens[:budget]
        return self.counter._get_encoder(self.model).decode(truncated_tokens)

    def _truncate_tail(self, text: str, budget: int):
        """Keep last N tokens"""
        tokens = self.counter._get_encoder(self.model).encode(text)
        truncated_tokens = tokens[-budget:]
        return self.counter._get_encoder(self.model).decode(truncated_tokens)

    def _truncate_middle(self, text: str, budget: int):
        """Keep beginning and end, remove middle"""
        tokens = self.counter._get_encoder(self.model).encode(text)

        if len(tokens) <= budget:
            return text

        # Keep first 40% and last 40% of budget
        head_budget = int(budget * 0.4)
        tail_budget = int(budget * 0.4)

        head_tokens = tokens[:head_budget]
        tail_tokens = tokens[-tail_budget:]

        # Add separator
        separator_tokens = self.counter._get_encoder(self.model).encode("\n.. [truncated

    combined_tokens = head_tokens + separator_tokens + tail_tokens
    return self.counter._get_encoder(self.model).decode(combined_tokens)

```

```
# Usage
manager = ContextWindowManager(model="gpt-3.5-turbo", max_tokens=4096)

components = {
    'system': {
        'text': "You are a helpful assistant specialized in Python programming.",
        'priority': 10 # Always include
    },
    'examples': {
        'text': "\n".join([f"Example {i}: ..." for i in range(5)]),
        'priority': 7
    },
    'context': {
        'text': "..." * 1000, # Large context
        'priority': 5,
        'truncation': 'middle' # Keep beginning and end
    },
    'history': {
        'text': "\n".join([f"User: Q{i}\nAssistant: A{i}" for i in range(10)]),
        'priority': 3,
        'truncation': 'tail' # Keep recent history
    },
    'user': {
        'text': "How do I implement a binary search tree?",
        'priority': 10 # Always include
    }
}

result = manager.allocate_budget(components, response_tokens=500)
print(f"Tokens used: {result['tokens_used']}/{result['tokens_available']}")
print(f"Utilization: {result['utilization']:.1%}")
```

Deliverable: Context manager that:

- Allocates tokens by priority
- Truncates gracefully when budget exceeded
- Provides multiple truncation strategies

Day 11-13: Prompt Compression

Hands-on Lab:

python

```

# src/prompt_compressor.py
import re

class PromptCompressor:
    def __init__(self, counter: TokenCounter):
        self.counter = counter

    def compress(self, text: str, target_reduction: float = 0.3) -> dict:
        """
        Compress prompt to reduce tokens

        target_reduction: 0.3 = 30% token reduction
        """
        original_tokens = self.counter.count_tokens(text)
        target_tokens = int(original_tokens * (1 - target_reduction))

        compressed = text
        techniques_used = []

        # Apply compression techniques in order
        compressed, removed = self._remove_redundancy(compressed)
        if removed > 0:
            techniques_used.append(f"redundancy_removal: -{removed} tokens")

        compressed, removed = self._condense_examples(compressed)
        if removed > 0:
            techniques_used.append(f"example_condensing: -{removed} tokens")

        compressed, removed = self._simplify_instructions(compressed)
        if removed > 0:
            techniques_used.append(f"instruction_simplification: -{removed} tokens")

        final_tokens = self.counter.count_tokens(compressed)
        actual_reduction = (original_tokens - final_tokens) / original_tokens

        return {
            'original': text,
            'compressed': compressed,
            'original_tokens': original_tokens,
            'final_tokens': final_tokens,
            'reduction': actual_reduction,
            'techniques': techniques_used
        }

    def _remove_redundancy(self, text: str) -> tuple:

```

```

        """Remove redundant phrases and repetition"""
        original_tokens = self.counter.count_tokens(text)

        # Remove filler words
        fillers = [
            'basically', 'actually', 'literally', 'very',
            'really', 'just', 'simply', 'certainly'
        ]
        for filler in fillers:
            text = re.sub(r'\b' + filler + r'\b', '', text, flags=re.IGNORECASE)

        # Remove excessive whitespace
        text = re.sub(r'\s+', ' ', text)
        text = re.sub(r'\n{3,}', '\n\n', text)

        # Remove redundant punctuation
        text = re.sub(r'\.{2,}', '.', text)
        text = re.sub(r'!\{2,}', '!', text)

        final_tokens = self.counter.count_tokens(text)
        return text.strip(), original_tokens - final_tokens

def _condense_examples(self, text: str) -> tuple:
    """Condense verbose examples"""
    original_tokens = self.counter.count_tokens(text)

    # Pattern: Convert verbose examples to concise format
    # Before: "For example, if you have X, then you would do Y because Z."
    # After: "Example: X → Y (Z)"

    patterns = [
        (r'For example, if you (.*)?, then you would (.*)? because (.*)?\.',
         r'Example: \1 → \2 (\3)'),
        (r'For instance, when (.*)?, you should (.*)?\.',
         r'When \1: \2'),
    ]

    for pattern, replacement in patterns:
        text = re.sub(pattern, replacement, text, flags=re.IGNORECASE)

    final_tokens = self.counter.count_tokens(text)
    return text, original_tokens - final_tokens

def _simplify_instructions(self, text: str) -> tuple:
    """Simplify verbose instructions"""
    original_tokens = self.counter.count_tokens(text)

```

```

# Simplify common verbose phrases
simplifications = {
    'in order to': 'to',
    'due to the fact that': 'because',
    'at this point in time': 'now',
    'make use of': 'use',
    'is able to': 'can',
    'has the ability to': 'can',
    'in the event that': 'if',
    'please ensure that you': 'ensure',
    'it is important to note that': '',
    'I would like you to': '',
}

for verbose, concise in simplifications.items():
    text = re.sub(
        r'\b' + re.escape(verbose) + r'\b',
        concise,
        text,
        flags=re.IGNORECASE
    )

final_tokens = self.counter.count_tokens(text)
return text, original_tokens - final_tokens

def compress_few_shot_examples(self, examples: list, max_tokens: int) -> list:
    """Select and compress examples to fit budget"""
    # Strategy: Keep most diverse examples
    from sentence_transformers import SentenceTransformer
    model = SentenceTransformer('all-MiniLM-L6-v2')

    embeddings = model.encode([ex['input'] for ex in examples])

    # Greedy selection for diversity
    selected = []
    selected_embeddings = []
    tokens_used = 0

    for i, ex in enumerate(examples):
        # Calculate diversity (distance from already selected)
        if selected_embeddings:
            similarities = [
                np.dot(embeddings[i], sel_emb)
                for sel_emb in selected_embeddings
            ]

```



```

        diversity = 1 - max(similarities)
    else:
        diversity = 1.0

    # Format example
    ex_text = f"Input: {ex['input']}\nOutput: {ex['output']}\n"
    ex_tokens = self.counter.count_tokens(ex_text)

    # Add if fits and diverse
    if tokens_used + ex_tokens <= max_tokens and diversity > 0.3:
        selected.append(ex_text)
        selected_embeddings.append(embeddings[i])
        tokens_used += ex_tokens

    return selected

# Usage
counter = TokenCounter()
compressor = PromptCompressor(counter)

verbose_prompt = """
You are a helpful assistant. It is important to note that you should basically
provide very clear and actually detailed explanations to users. When users ask
questions, you should really make use of examples in order to help them understand.

For example, if you have a user asking about Python, then you would explain Python
features because that helps them learn better.
"""

result = compressor.compress(verbose_prompt, target_reduction=0.4)
print(f"Original: {result['original_tokens']} tokens")
print(f"Compressed: {result['final_tokens']} tokens")
print(f"Reduction: {result['reduction']:.1%}")
print(f"\nTechniques:")
for tech in result['techniques']:
    print(f"    - {tech}")
print(f"\nCompressed text:\n{result['compressed']}")

```

Deliverable: Compression system achieving:

- 30-50% token reduction
- Preserved semantic meaning (measure with eval)
- Documented compression techniques

Day 14: Quality-Preserving Optimization

Hands-on Lab:

Test that compression doesn't hurt quality:

python

```
# tests/test_compression_quality.py
class CompressionQualityTester:
    def __init__(self, test_cases):
        self.test_cases = test_cases
        self.compressor = PromptCompressor(TokenCounter())

    def test_compression(self, reduction_levels=[0.2, 0.3, 0.4, 0.5]):
        """Test compression at different levels"""
        results = []

        for level in reduction_levels:
            for test in self.test_cases:
                # Compress prompt
                compressed = self.compressor.compress(
                    test['prompt'],
                    target_reduction=level
                )

                # Get responses
                response_original = call_llm(test['prompt'])
                response_compressed = call_llm(compressed['compressed'])

                # Compare quality
                quality_score = self.evaluate_quality(
                    test['expected_output'],
                    response_original,
                    response_compressed
                )

                results.append({
                    'reduction_level': level,
                    'tokens_saved': compressed['original_tokens'] - compressed['final_tokens'],
                    'quality_score': quality_score,
                    'acceptable': quality_score > 0.85 # 85% threshold
                })

        return pd.DataFrame(results)
```

```

def evaluate_quality(self, expected, original_response, compressed_response):
    """Evaluate if compressed response maintains quality"""
    # Simple similarity check
    # Production: Use more sophisticated metrics (BLEU, ROUGE, etc.)
    from difflib import SequenceMatcher

    original_sim = SequenceMatcher(None, expected, original_response).ratio()
    compressed_sim = SequenceMatcher(None, expected, compressed_response).ratio()

    return compressed_sim / original_sim if original_sim > 0 else 0

# Run tests
test_cases = [
    {
        'prompt': "Explain Python decorators...",
        'expected_output': "Decorators are..."
    },
    # ... more test cases
]

tester = CompressionQualityTester(test_cases)
results = tester.test_compression()

print(results.groupby('reduction_level').agg({
    'tokens_saved': 'mean',
    'quality_score': 'mean',
    'acceptable': 'mean'
})))

# Output:
# reduction_level  tokens_saved  quality_score  acceptable
# 0.2              45.2          0.96           1.00
# 0.3              68.1          0.91           0.95
# 0.4              90.5          0.83           0.75
# 0.5             113.2          0.72           0.45
#
# Conclusion: 30% reduction optimal (91% quality, 95% acceptance rate)

```

Deliverable: Quality analysis showing:

- Compression level vs. quality tradeoff
 - Optimal compression target
 - Cost savings calculation
-

Week 3: Production Optimization & Portfolio

Learning Objectives:

- Implement model routing
- Build batch processing pipeline
- Create production-ready cost optimization system

Day 15-17: Intelligent Model Routing

Hands-on Lab:

python

```
# src/model_router.py
class IntelligentModelRouter:
    def __init__(self, counter: TokenCounter, tracker: CostTracker):
        self.counter = counter
        self.tracker = tracker

    # Model capabilities and costs
    self.models = {
        'gpt-4': {
            'cost_per_1k_in': 0.03,
            'cost_per_1k_out': 0.06,
            'quality_score': 1.0,
            'max_tokens': 8192
        },
        'gpt-3.5-turbo': {
            'cost_per_1k_in': 0.0015,
            'cost_per_1k_out': 0.002,
            'quality_score': 0.85,
            'max_tokens': 4096
        }
    }

    def estimate_complexity(self, query: str) -> float:
        """Estimate query complexity (0-1)"""
        factors = []

        # 1. Length
        word_count = len(query.split())
        factors.append(min(word_count / 50, 1.0))

        # 2. Technical indicators
```

```

technical_terms = [
    'implement', 'design', 'architecture', 'algorithm',
    'optimize', 'debug', 'analyze', 'compare'
]
tech_score = sum(1 for term in technical_terms if term in query.lower())
factors.append(min(tech_score / 3, 1.0))

# 3. Multi-step reasoning
step_indicators = ['first', 'then', 'after', 'finally', 'because']
step_score = sum(1 for ind in step_indicators if ind in query.lower())
factors.append(min(step_score / 2, 1.0))

# 4. Code indicators
code_indicators = ['`', 'def ', 'class ', 'import ', 'function']
code_score = sum(1 for ind in code_indicators if ind in query)
factors.append(min(code_score / 2, 1.0))

return np.mean(factors)

def route_query(self, query: str, context: str = "", user_id: str = None) -> dict:
    """Route query to optimal model"""
    # Calculate complexity
    complexity = self.estimate_complexity(query)

    # Calculate total tokens
    total_text = f"{context}\n{query}"
    tokens = self.counter.count_tokens(total_text)

    # Routing logic
    if tokens > 4000:
        # Must use larger context window
        model = 'gpt-4'
        reason = f"Context size ({tokens} tokens) requires large window"

    elif complexity > 0.7:
        # Complex query needs GPT-4
        model = 'gpt-4'
        reason = f"High complexity ({complexity:.2f})"

    elif complexity < 0.3:
        # Simple query can use GPT-3.5
        model = 'gpt-3.5-turbo'
        reason = f"Low complexity ({complexity:.2f})"

    else:
        # Medium complexity: try GPT-3.5 with fallback

```

```

        model = 'gpt-3.5-turbo-with-fallback'
        reason = f"Medium complexity ({complexity:.2f}), trying cheap first"

    return {
        'model': model,
        'complexity': complexity,
        'tokens': tokens,
        'reason': reason
    }

def complete_with_routing(self, query: str, context: str = "", user_id: str = None):
    """Complete with automatic model selection"""
    routing = self.route_query(query, context, user_id)
    model = routing['model']

    if model == 'gpt-3.5-turbo-with-fallback':
        try:
            # Try GPT-3.5 first
            response = self._call_llm('gpt-3.5-turbo', query, context)

            # Check if response seems incomplete/low-quality
            if self._is_low_quality(response):
                logger.warning("GPT-3.5 response low quality, escalating to GPT-4")
                response = self._call_llm('gpt-4', query, context)
                model = 'gpt-4'
            else:
                model = 'gpt-3.5-turbo'

        except Exception as e:
            logger.error(f"GPT-3.5 failed: {e}, falling back to GPT-4")
            response = self._call_llm('gpt-4', query, context)
            model = 'gpt-4'

    else:
        response = self._call_llm(model, query, context)

    # Track cost
    input_tokens = self.counter.count_tokens(f"{context}\n{query}", model)
    output_tokens = self.counter.count_tokens(response, model)

    cost = (
        input_tokens / 1000 * self.models[model]['cost_per_1k_in'] +
        output_tokens / 1000 * self.models[model]['cost_per_1k_out']
    )

    self.tracker.log_request(

```

```

        model=model,
        input_tokens=input_tokens,
        output_tokens=output_tokens,
        cost=cost,
        user_id=user_id
    )

    return {
        'response': response,
        'model_used': model,
        'routing_reason': routing['reason'],
        'cost': cost
    }

def _is_low_quality(self, response: str) -> bool:
    """Heuristic to detect low-quality responses"""
    # Check for common failure patterns
    failure_patterns = [
        "I don't have enough information",
        "Could you clarify",
        "I'm not sure",
        len(response) < 50 # Very short response
    ]

    return any(
        pattern in response if isinstance(pattern, str) else pattern
        for pattern in failure_patterns
    )

def _call_llm(self, model: str, query: str, context: str = ""):
    """Call LLM API"""
    # Implementation details...
    pass

# Usage
router = IntelligentModelRouter(counter, tracker)

# Simple query
result = router.complete_with_routing("What is Python?")
print(f"Model: {result['model_used']}, Cost: ${result['cost']:.4f}")
# Output: Model: gpt-3.5-turbo, Cost: $0.0008

# Complex query
result = router.complete_with_routing(
    "Design a distributed system for processing 1M events/second with exactly once semantic"
)

```

```
print(f"Model: {result['model_used']], Cost: ${result['cost']:.4f}")
# Output: Model: gpt-4, Cost: $0.0450
```

Deliverable: Routing system that:

- Selects models based on complexity
- Falls back intelligently on failures
- Reduces costs by 60-80%

Day 18-19: Batch Processing

Hands-on Lab:

python

```
# src/batch_processor.py
class BatchProcessor:
    def __init__(self, batch_size=20, max_wait_time=5):
        self.batch_size = batch_size
        self.max_wait_time = max_wait_time
        self.queue = []
        self.results = {}

    def submit(self, request_id: str, prompt: str, callback=None):
        """Submit request to batch"""
        self.queue.append({
            'id': request_id,
            'prompt': prompt,
            'callback': callback,
            'submitted_at': time.time()
        })

        # Process if batch full or timeout
        if len(self.queue) >= self.batch_size:
            self._process_batch()

        return request_id

    def _process_batch(self):
        """Process accumulated batch"""
        if not self.queue:
            return

        batch = self.queue[:self.batch_size]
        self.queue = self.queue[self.batch_size:]
```



```

# Call batch API
prompts = [req['prompt'] for req in batch]

# OpenAI batch example (using embeddings API for demonstration)
# For completions, use OpenAI Batch API
responses = self._call_batch_api(prompts)

# Store results and trigger callbacks
for req, response in zip(batch, responses):
    self.results[req['id']] = response
    if req['callback']:
        req['callback'](response)

def _call_batch_api(self, prompts: list):
    """Call batch API - placeholder"""
    # Production: Use actual batch API
    # OpenAI: https://platform.openai.com/docs/guides/batch
    return [f"Response to: {p[:50]}" for p in prompts]

def get_result(self, request_id: str, timeout=30):
    """Get result for request (blocking)"""
    start = time.time()

    while request_id not in self.results:
        if time.time() - start > timeout:
            raise TimeoutError(f"Request {request_id} timed out")

        # Check if should process batch (timeout)
        if self.queue:
            oldest = min(self.queue, key=lambda x: x['submitted_at'])
            if time.time() - oldest['submitted_at'] > self.max_wait_time:
                self._process_batch()

        time.sleep(0.1)

    return self.results[request_id]

# Usage
processor = BatchProcessor(batch_size=10)

# Submit multiple requests
request_ids = []
for i in range(25):
    req_id = processor.submit(
        f"req_{i}",

```

```

        f"Summarize: Document {i}",
        callback=lambda r: print(f"Got result: {r}")
    )
    request_ids.append(req_id)

# Batching automatically happens:
# - First 10 processed immediately (batch full)
# - Next 10 processed immediately (batch full)
# - Last 5 processed after max_wait_time

# Get results
for req_id in request_ids[:3]:
    result = processor.get_result(req_id)
    print(f"{req_id}: {result}")

# Cost savings:
# - Individual requests: 25 × $0.002 = $0.05
# - Batched (50% discount): 25 × $0.001 = $0.025
# - Savings: 50%

```

Deliverable: Batch processing achieving:

- 50% cost reduction (batch API discount)
- <10s latency for non-urgent requests
- Automatic batch assembly

Day 20-21: Portfolio & Documentation

Create comprehensive portfolio:

markdown

```

# LLM Cost Optimization System

## Overview
Production-ready cost optimization system reducing LLM API costs by 75% while maintainin

## Architecture

```

Request → Complexity Analysis → Model Router → Batch Processor → Response

↓↓↓

Cost Tracker Context Manager Cache Layer

Results

Cost Reduction

Optimization	Savings	Implementation
-----	-----	-----
Model routing	65%	Route simple→GPT-3.5, complex→GPT-4
Prompt compression	35%	Remove redundancy, condense examples
Batch processing	50%	Group requests, use batch API
Response caching	80%	Cache deterministic queries
Combined	**87%**	All techniques together

Real-World Performance

****Before optimization:****

- 100k requests/day
- 500 avg input tokens, 300 avg output tokens
- GPT-4 for all requests
- Cost: \$2,400/day (\$72k/month)

****After optimization:****

- Same 100k requests/day
- 350 avg input tokens (30% compression)
- 20% GPT-4, 80% GPT-3.5 (routing)
- Batch processing (50% discount)
- 40% cache hit rate
- Cost: \$312/day (\$9.4k/month)

****Savings: \$62.6k/month (87% reduction)****

Features

1. Token-Accurate Cost Tracking

```
```python
tracker = CostTracker()
tracker.log_request(model="gpt-4", input_tokens=500, output_tokens=300, cost=0.033)

stats = tracker.get_stats(group_by='user_id')
{'total_cost': 45.67, 'top_user': 'user123' ($12.34)}
```

## 2. Smart Model Routing

```
router = IntelligentModelRouter()
result = router.complete_with_routing("What is Python?")
Automatically uses GPT-3.5 ($0.0008 vs $0.03 for GPT-4)
```

## 3. Context Window Management

```
manager = ContextWindowManager(max_tokens=4096)
result = manager.allocate_budget(components, response_tokens=500)
Fits all components within budget via priority-based truncation
```

## 4. Prompt Compression

```
compressor = PromptCompressor()
result = compressor.compress(verbose_prompt, target_reduction=0.3)
30% token reduction while preserving quality
```

## Quality Metrics

Tested on 1000 diverse queries:

Metric	Before	After	Change
Accuracy	94.2%	93.8%	-0.4% ✓
Latency (p95)	2.1s	2.4s	+0.3s ✓
Cost/query	\$0.024	\$0.003	-87.5% ✓

**Conclusion:** 87% cost reduction with <1% quality degradation.

## Usage

---

```
Install
pip install -r requirements.txt

Run optimization
python src/main.py --input queries.jsonl --output results.jsonl

View cost dashboard
python src/dashboard.py
```

## Design Decisions

---

### Why Model Routing?

GPT-4 is 20x more expensive than GPT-3.5 but only 10-15% better on simple tasks. Routing saves 60-70% of costs with minimal quality impact.

**Tradeoff:** Adds complexity (routing logic), but worth it at scale.

### Why Prompt Compression?

Every token costs money. 30% compression = 30% savings.

**Tradeoff:** Requires testing to ensure quality preserved.

### Why Batch Processing?

Batch API offers 50% discount. For non-urgent requests, batching is free money.

**Tradeoff:** Adds latency (wait for batch to fill), acceptable for async workloads.

## Production Monitoring

---

- Real-time cost dashboard (Grafana)
- Per-user cost attribution
- Automated budget alerts
- Quality regression detection

## Future Work

---

1. Fine-tune routing classifier on real data (+5% accuracy)
2. Implement prompt caching (additional 50% savings)
3. Add semantic caching (similar queries reuse responses)

...

...

---

## 4. Blind Spots & Underrated Perspectives

### What's Poorly Taught (Industry Reality Gaps)

#### Blind Spot 1: Token Counting Seems Simple But Is Model-Specific

**Common teaching:** "Count words and multiply by 1.3 to estimate tokens."

**Reality:** Tokenization varies dramatically by model, language, and content type.

**Examples:**

python

```
import tiktoken

text = "Hello, world!"

Different models, different token counts
gpt4_enc = tiktoken.encoding_for_model("gpt-4")
gpt35_enc = tiktoken.encoding_for_model("gpt-3.5-turbo")

print(f"GPT-4: {len(gpt4_enc.encode(text))} tokens")
print(f"GPT-3.5: {len(gpt35_enc.encode(text))} tokens")

Different content types
english = "The quick brown fox jumps" * 10
chinese = "快速的棕色狐狸跳跃" * 10
code = "def foo():\n return True\n" * 10

print(f"English: {len(gpt4_enc.encode(english))} tokens") # ~50
```

```
print(f"Chinese: {len(gpt4_enc.encode(chinese))} tokens") # ~100 (2x!)
print(f"Code: {len(gpt4_enc.encode(code))} tokens") # ~80

Tokens per character varies wildly
print(f"English: {len(english)/len(gpt4_enc.encode(english)):.2f} chars/token") # ~4.0
print(f"Chinese: {len(chinese)/len(gpt4_enc.encode(chinese)):.2f} chars/token") # ~2.0
```

### Production impact:

- Switching from GPT-3.5 to GPT-4 can change token counts by 10-15%
- Non-English content costs 2-3x more (more tokens per character)
- Code uses more tokens than prose (special characters fragment more)

### How strong engineers handle this:

python

```
class ModelAwareTokenCounter:
 def count_with_model_adjustment(self, text, source_model, target_model):
 """Account for tokenization differences when switching models"""
 source_count = self.count_tokens(text, source_model)

 # Empirical adjustment factors (measure on your data)
 adjustments = {
 ('gpt-3.5-turbo', 'gpt-4'): 1.12, # GPT-4 uses ~12% more tokens
 ('gpt-4', 'gpt-3.5-turbo'): 0.89,
 }

 adjustment = adjustments.get((source_model, target_model), 1.0)
 estimated_target_count = int(source_count * adjustment)

 return estimated_target_count
```

## Blind Spot 2: Context Windows Aren't Just About Maximum Tokens

**Common teaching:** "GPT-4 has 8k context, Claude has 100k context."

**Reality:** Effective context windows are much smaller due to:

1. **Attention degradation** (quality drops with distance)
2. **Cost scaling** (larger context = higher cost)
3. **Latency impact** (more tokens = slower generation)
4. **The "lost in the middle" problem**

## Research finding: LLMs perform worst on information in the middle of context

python

```
Study: Place key info at different positions in 8k token context
positions = [0, 2000, 4000, 6000, 8000] # Beginning to end

Retrieval accuracy by position
accuracies = [0.95, 0.75, 0.60, 0.78, 0.93]
^^^^ ^^^^ ^^^^ ^^^^ ^^^^
Start Near Mid Near End
(worst!)

Takeaway: Information in middle gets "lost"
```

## How strong engineers work around this:

python

```
class SmartContextPlacement:
 def arrange_context(self, system_prompt, examples, rag_results, query):
 """Place most important info at beginning and end"""

 # Structure (avoids middle placement):
 prompt_parts = [
 system_prompt, # Beginning (high attention)
 query, # Beginning (user intent clear)
 "\n\nRelevant Context:",
 rag_results[0], # Most relevant at beginning
 "\n\n...\n\n", # Middle (less important examples)
 *examples[1:-1],
 "\n\n...\n\n",
 rag_results[-1], # Second most relevant at end
 "\n\nBased on the above context, " + query # End (high attention)
]

 return "\n".join(prompt_parts)
```

## Cost implications of large contexts:

python

```
Claude 100k context looks cheap at first glance
$0.015/1k input tokens
```



```
But:
request_1 = {
 'input_tokens': 100_000,
 'output_tokens': 1_000,
 'cost': 100 * 0.015 + 1 * 0.075 # $1.575 per request
}

For comparison, GPT-4 8k:
request_2 = {
 'input_tokens': 8_000,
 'output_tokens': 1_000,
 'cost': 8 * 0.03 + 1 * 0.06 # $0.30 per request
}

100k context is 5x more expensive despite lower per-token cost!
```

**Strong engineer perspective:** "Context window is a budget to spend wisely, not a garage to dump everything into."

### Blind Spot 3: Output Tokens Are More Expensive (and Harder to Control)

**Common teaching:** Focus on input optimization.

**Reality:** Output tokens often cost 2-3x more and are harder to predict/control.

**Pricing asymmetry:**

python

```
GPT-4 pricing
input_cost = 0.03 # per 1k tokens
output_cost = 0.06 # per 1k tokens (2x more expensive!)

Claude Opus
input_cost = 0.015 # per 1k tokens
output_cost = 0.075 # per 1k tokens (5x more expensive!)

For verbose responses, output dominates cost
verbose_request = {
 'input': 100 tokens,
 'output': 2000 tokens,
 'cost': (100 * 0.03 + 2000 * 0.06) / 1000
}

print(f"Input cost: ${100 * 0.03 / 1000:.4f}") # $0.0030
```

```
print(f"Output cost: ${2000 * 0.06 / 1000:.4f}") # $0.1200
Output is 40x the cost of input!
```

### The max\_tokens trap:

python

```
Amateur: Set max_tokens high "to be safe"
response = llm.complete(prompt, max_tokens=4000)
Problem: LLM often fills the budget even if not needed

Engineer: Set max_tokens based on expected output
def calculate_optimal_max_tokens(task_type):
 # Based on empirical measurements
 optimal_lengths = {
 'classification': 10,
 'extraction': 50,
 'short_summary': 150,
 'long_summary': 500,
 'code_generation': 1000,
 'essay': 2000
 }
 return optimal_lengths.get(task_type, 500)

max_tokens = calculate_optimal_max_tokens('short_summary')
response = llm.complete(prompt, max_tokens=max_tokens)
Saves money by not over-allocating
```

### Advanced: Use stop sequences to control output length:

python

```
Instead of hoping LLM stops naturally
response = llm.complete(
 prompt="List 5 Python tips:\n1.",
 max_tokens=500 # Safety limit
)
Problem: Might generate 10 tips (wastes tokens)

Better: Use stop sequences
response = llm.complete(
 prompt="List 5 Python tips:\n1.",
 max_tokens=500,
 stop=["\n6.", "\n\n"] # Stop after 5 items
```

```
)
Guaranteed to stop after 5 tips
```

## Blind Spot 4: Streaming Reduces Latency But Complicates Cost Tracking

**Common teaching:** "Use streaming for better UX."

**Reality:** Streaming requires different cost tracking and error handling.

**The streaming cost problem:**

python

```
Non-streaming: Easy cost tracking
response = llm.complete(prompt)
cost = (
 response.usage.prompt_tokens * price_per_token_in +
 response.usage.completion_tokens * price_per_token_out
)

Streaming: No usage info until end
stream = llm.complete(prompt, stream=True)
for chunk in stream:
 print(chunk) # How much is this costing so far? Unknown!

Must track tokens manually
total_output_tokens = 0
for chunk in stream:
 total_output_tokens += count_tokens(chunk)
 print(chunk)

Calculate cost after stream completes
cost = (input_tokens * price_in + total_output_tokens * price_out)
```

**Advanced streaming cost control:**

python

```
class CostAwareStreamHandler:
 def __init__(self, budget_limit=0.10):
 self.budget_limit = budget_limit
 self.tokens_generated = 0
 self.cost_so_far = 0

 def stream_with_budget(self, prompt):
```

```

"""Stream response but stop if budget exceeded"""
input_tokens = count_tokens(prompt)
input_cost = input_tokens / 1000 * 0.03

if input_cost > self.budget_limit:
 raise BudgetExceededError("Input alone exceeds budget")

stream = openai.ChatCompletion.create(
 model="gpt-4",
 messages=[{"role": "user", "content": prompt}],
 stream=True
)

accumulated = []
for chunk in stream:
 delta = chunk.choices[0].delta.content
 if delta:
 accumulated.append(delta)
 self.tokens_generated += count_tokens(delta)

 # Calculate current cost
 self.cost_so_far = (
 input_cost +
 self.tokens_generated / 1000 * 0.06
)

 # Check budget
 if self.cost_so_far > self.budget_limit:
 logger.warning(f"Budget exceeded at {self.tokens_generated} tokens")
 break # Stop streaming

 yield delta

return ''.join(accumulated)

```

## Blind Spot 5: Caching is 90% Cost Savings But Rarely Implemented

**Common teaching:** Not mentioned at all.

**Reality:** Prompt caching (where available) is the single highest-ROI optimization.

**Anthropic Claude caching example:**

python

```

Without caching: Every request pays for system prompt
for i in range(1000):
 response = claude.complete(
 system=LONG_SYSTEM_PROMPT, # 5000 tokens
 user=user_queries[i] # 100 tokens
)
 # Cost per request: (5000 + 100) * $0.003 / 1000 = $0.0153
Total: 1000 * $0.0153 = $15.30

With caching: Only first request pays for system prompt
response = claude.complete(
 system=[
 {"text": LONG_SYSTEM_PROMPT, "cache_control": {"type": "ephemeral"}}
],
 user=user_queries[0]
)
First request: (5000 * $0.003 + 100 * $0.003) / 1000 = $0.0153

for i in range(1, 1000):
 response = claude.complete(
 system=[
 {"text": LONG_SYSTEM_PROMPT, "cache_control": {"type": "ephemeral"}}
],
 user=user_queries[i]
)
 # Subsequent requests:
 # Cached: 5000 tokens * $0.0003 / 1000 = $0.0015
 # New: 100 tokens * $0.003 / 1000 = $0.0003
 # Total: $0.0018

Total cost: $0.0153 + (999 * $0.0018) = $1.80
Savings: $15.30 → $1.80 = 88% reduction!

```

### Application-level caching for models without native support:

python

```

import hashlib
import redis

class SemanticCache:
 def __init__(self, similarity_threshold=0.95):
 self.cache = redis.Redis()
 self.threshold = similarity_threshold

```

```

self.embedder = SentenceTransformer('all-MiniLM-L6-v2')

def get_cached_response(self, prompt):
 """Check if similar prompt has been cached"""
 prompt_emb = self.embedder.encode([prompt])[0]

 # Search for similar cached prompts
 cached_prompts = self.cache.keys("prompt:*")

 for cached_key in cached_prompts:
 cached_emb = np.frombuffer(
 self.cache.get(f"{cached_key}:embedding"),
 dtype=np.float32
)

 similarity = np.dot(prompt_emb, cached_emb)

 if similarity > self.threshold:
 # Cache hit!
 response = self.cache.get(f"{cached_key}:response")
 return response.decode()

 return None

def cache_response(self, prompt, response):
 """Cache prompt and response"""
 prompt_hash = hashlib.sha256(prompt.encode()).hexdigest()
 prompt_emb = self.embedder.encode([prompt])[0]

 self.cache.set(f"prompt:{prompt_hash}", prompt)
 self.cache.set(f"prompt:{prompt_hash}:embedding", prompt_emb.tobytes())
 self.cache.set(f"prompt:{prompt_hash}:response", response)
 self.cache.expire(f"prompt:{prompt_hash}", 86400) # 24h TTL

Usage
cache = SemanticCache()

First query
response = cache.get_cached_response("What is Python?")
if not response:
 response = llm.complete("What is Python?")
 cache.cache_response("What is Python?", response)

Similar query (cache hit!)
response = cache.get_cached_response("Can you explain Python?")
Returns cached response (95% similar), saves API call

```

---

## How This Perspective Prevents Failures

### Fragile System (Amateur):

- No token counting (guesses costs)
- Single model for everything (GPT-4 for simple tasks)
- No context window management (frequent overflows)
- No output control (verbose responses waste money)
- No caching (repeat calls for same content)
- **Result:** \$10k/month → \$100k/month as usage grows, no idea why

### Robust System (Engineer):

- Accurate token counting per model
- Intelligent routing (GPT-3.5 for 80% of queries)
- Priority-based context allocation
- Output length control via stop sequences
- Multi-layer caching (exact + semantic)
- **Result:** \$10k/month → \$15k/month at 10x usage (costs scale sublinearly)